SYNERGETICS
GET IT RIGHT

Java 8 New Features

Makarand Bhoir

Microsoft Partner | Silver Learning Gold Cloud Platform

Microsoft Cloud Solution Provider

Educate | Advise | Implement | Manage

Agenda

- Understanding Function Interface
- Lambda Expression
- Method Reference
- Stream API
- Date Time API

Interface

- In general, when we create an interface, we are defining a contract for what a class can do; without saying anything about how the class will do it.
- A class “signs” the contract with the keyword implements
- When implementing an interface, you are agreeing to adhere (obey) to the contract defined in the interface.
- If a concrete (non abstract) class is implementing an interface, the compiler will ensure that the class has implementation code for each abstract method in the interface.

Interface

- Whereas a class can extend from only one other class , a class can implement many interfaces.
- As of Java 8, it is now possible to inherit concrete methods from interfaces. Interfaces can now contain two types of concrete methods: static and default
- Implementation classes are NOT required to implement an interface's static or default methods. The default interface methods are inheritable but the static interface methods are not.

Interface

- Interfaces themselves are implicitly abstract
- public abstract interface I{}
- top level interfaces can have public or package private access
- All interface methods are implicitly public

Interface

- All interface methods are implicitly abstract (unless declared as default or static , the new features introduced in Java 8).
- All variables declared in an interface must be public ,static and final i.e., interfaces can only declare constants (not instance variables).
- As with abstract classes you cannot new an interface type, but they can be used as references:

```
Printable p = new Printer(); // Printable is an interface
```

Functional Interface

SYNERGETICS
GET IT RIGHT

- A functional interface is an interface that has only one abstract method. This is known as the SAM (Single Abstract Method) rule.
- default methods do not count
- static methods do not count
- methods inherited from Object do not count

Lambdas

- A lambda expression is just a block of code that helps in making your code more concise.
- A lambda expression only works with functional interfaces.
- A lambda expression is an instance of a class that implements a functional interface

Lambdas

- Lambdas look a lot like methods and in some quarters are called “anonymous methods”. However, it is an instance with everything, but the method stripped away
- A lot can be inferred (by the compiler) from the interface definition (which remember, has only one abstract method). The lambda expression is the instance that implements the interface that has been boiled down to the bare essentials.

Lambdas

```
interface I{
    void m();// a functional interface i.e. it has only one
             // abstract method
}

public class BasicLambdas {
    public static void main(String[] args) {
        // pre-Java 8
        I i = new I(){
            @Override
            public void m(){
                System.out.println("I::m()");
            }
        };
        i.m();  //  I::m()

        // Java 8 - Lambda expression
        I lambdaI = () ->  {
            System.out.println("Lambda version");
        };
        I lambdaI2 = () ->  System.out.println("Lambda version");
        lambdaI.m();   //  Lambda version
        lambdaI2.m();  //  Lambda version
```

Method code.

SYNERGETICS
GET IT RIGHT

Method Reference

Educate | Advise | Implement | Manage

Method Reference

SYNERGETICS
GET IT RIGHT

- Lambda expressions help in making your code more concise (fewer keystrokes).
- Method references, can, in certain situations, help in making your lambda expressions even more concise.
- If all your lambda expression does is call one method, then that is an opportunity to use a method reference.
- If a lambda parameter is simply passed to another method, then the redundancy of specifying the variable twice can be removed.

Method Reference

SYNERGETICS
GET IT RIGHT

- There are four different styles/types:
  - ✓ Bound instance known at compile time
  - ✓ Unbound instance provided at runtime
  - ✓ Static
  - ✓ Constructor
- With method references, context is key!
- the functional interface type being created is hugely important in determining context.
- The compiler turns your method references into lambdas in the background.

SYNERGETICS

GET IT RIGHT

Stream API

Educate | Advise | Implement | Manage

What is Stream?

- A stream is a sequence of data that can be processed with operations.
- Streams are not another way of organizing data, like an array or a Collection
- Streams do not hold data; streams are all about processing data efficiently.

What is Stream?

- While streams make code more concise, their big advantage is that streams, by using a pipeline, can, in certain situations, greatly improve the efficiency of data processing.
- The real power of streams comes from the multiple intermediate operations you can perform on the stream.

Pipeline

- A stream pipeline consists of the operations that run on a stream to produce a result.
- There are 3 parts to a stream pipeline:
  - Source where the stream comes from e.g., array, collection or file.
  - Intermediate operations transforms the stream into another one. There can as few or as many as required.
  - Terminal operation required to start the whole process and produces the result. Streams can only be used once i.e.; streams are no longer usable after a terminal operation completes (re generate the stream if necessary).

Pipeline

- filter() is an intermediate operation and as such can filter the stream and pass on the filtered stream to the next operation (another intermediate operation or a terminal operation
- count() and forEach are both terminal operations that end the stream
- The pipeline operations are the way in which we specify how and in what order we want the data in the source manipulated. Remember, streams don't hold any data.

Pipeline

- filter() is an intermediate operation and as such can filter the stream and pass on the filtered stream to the next operation (another intermediate operation or a terminal operation
- count() and forEach are both terminal operations that end the stream
- The pipeline operations are the way in which we specify how and in what order we want the data in the source manipulated. Remember, streams don't hold any data.

Demo

- PipelineProcessingOrder.java

Stream Are Lazy

- The principle of “lazy” evaluation is that you get what you need only when you need it. For example, if you were displaying 10,000 records to a user, the principle of lazy evaluation would be to retrieve 50 and while the user is viewing these, retrieve another 50 in the background.
- “Eager” evaluation would be to retrieve all 10,000 records in one go.
- With regard to streams, this means that nothing happens until the terminal operation occurs.

Stream Are Lazy

- The pipeline specifies what operations we want performed (on the source) and in which order.
- This enables the JDK to reduce operations whenever possible.
- For example, why run an operation on a piece of data if the operation is not required:
  - we have found the data element we were looking for
  - we may have a limit set on the number of elements we want to operate on

Demo

- SreamAreLazy.java demo

Creating stream from an array

- Arrays.stream can be used to stream an array.

```
Double[] numbers = {1.1, 2.2, 3.3};
// Arrays.stream() creates a stream from the array 'numbers'.
// The array is considered the source of the stream and while the
// data is flowing through the stream, we have an opportunity to
// operate on the data.
Stream<Double> stream1 = Arrays.stream(numbers);

// lets perform an operation on the data
// note that count() is a "terminal operation" - this means that
// you cannot perform any more operations on the stream.
long n = stream1.count();
System.out.println("Number of elements: "+n);// 3
```

Creating stream from collection

SYNERGETICS
GET IT RIGHT

- The default Collection interface method stream() is used.

```
List<String> animalList = Arrays.asList("cat", "dog", "sheep");
// using stream() which is a default method in Collection interface
Stream<String> streamAnimals = animalList.stream();
System.out.println("Number of elements: "+streamAnimals.count()); // 3

// stream() is a default method in the Collection interface and therefore
// is inherited by all classes that implement Collection. Map is NOT one
// of those i.e. Map is not a Collection. To bridge between the two, we
// use the Map method entrySet() to return a Set view of the Map (Set
// IS-A Collection).
Map<String, Integer> namesToAges = new HashMap<>();
namesToAges.put("Mike", 22);namesToAges.put("Mary", 24);namesToAges.put("Alice", 31);
System.out.println("Number of entries: "+
        namesToAges
            .entrySet() // get a Set (i.e. Collection) view of the Map
            .stream()   // stream() is a default method in Collection
            .count());  // 3
```

Creating stream with Stream.of

- Stream.of is a static generically typed utility method that accepts a varargs parameter and returns an ordered stream of those values.

Creating stream from a File

- The Files.lines method can be used to stream a file . It provides one line at a time from the file as a data element in the stream.
- To process the data from the stream, we use the Stream interfaces' forEach method, which is a terminal operation.
- Similar to the forEach () for collections, it takes a Consumer, which enables us to process each line from the file.

Demo

- SreamFromFiles.java demo

Terminal Operations

- Terminal operations can be performed without any intermediate operations but not the other way around
- Reductions are a special type of terminal operation where all of the contents of the stream are combined into a single primitive or Object (Collection).
- We will have a look at the most common ones over the coming slides. Note that the comments in the code are very important.

Terminal Operations

SYNERGETICS
GET IT RIGHT

Method	Return value	Reduction[1]
count()	long	Yes
min(), max()	Optional<T> - stream may be empty	Yes
findAny(), findFirst()	Optional<T>	No – may not look at all of the elements
allMatch(), anyMatch(), noneMatch()	boolean	No – may not look at all of the elements
forEach()	void	No (as it does not return anything)
reduce()	varies	Yes
collect()	varies	Yes

[1] Reductions are a special type of terminal operation where ALL of the contents of the stream are combined into a single primitive or Object e.g. long or Collection.

Intermediate Operations - filter

SYNERGETICS
GET IT RIGHT

- Unlike a terminal operation, an intermediate operation produces a stream as a result.

```
// Stream<T> filter(Predicate)
// The filter() method returns a Stream with the elements that
// MATCH the given predicate.
Stream.of("galway", "mayo", "roscommon")
        .filter(countyName -> countyName.length() > 5)
        .forEach(System.out::print);// galwayroscommon
```

Intermediate Operations - distinct

SYNERGETICS
GET IT RIGHT

- distinct() returns a stream with duplicate values removed.
- equals () is used i.e. case sensitive
- distinct() is a stateful intermediate operation.
- it behaves like a filter if it has not seen the object previously, it passes it on and remembers it; if it has seen it already, it filters it out.

Intermediate Operations - distinct

```
// Stream<T> distinct()
// distinct() is a stateful intermediate operation
// Output: 1.eagle 2.eagle 1.eagle 1.EAGLE 2.EAGLE
Stream.of("eagle", "eagle", "EAGLE")
        .peek(s -> System.out.print(" 1."+s))
        .distinct()
        .forEach(s -> System.out.print(" 2."+s));
```

Intermediate Operations - limit

- limit() is a short circuiting stateful intermediate operation.

```
// Stream<T> limit(long maxSize)
// limit is a short-circuiting stateful
// intermediate operation. Lazy evaluation - 66, 77, 88 and 99
// are not streamed as they are not needed (limit of 2 i.e. 44 and 55)
// Output:
//  A - 11 A - 22 A - 33 A - 44 B - 44 C - 44 A - 55 B - 55 C - 55
Stream.of(11,22,33,44,55,66,77,88,99)
        .peek(n -> System.out.print(" A - "+n))
        .filter(n -> n > 40)
        .peek(n -> System.out.print(" B - "+n))
        .limit(2)
        .forEach(n -> System.out.print(" C - "+n));
```

Intermediate Operations - map

- map() creates a one to one mapping between elements in the stream and elements in the next stage of the stream.
- map () is for transforming data.

```
// <R> Stream<R> map(Function<T,R> mapper)
//      Function's functional method: R apply(T t);
Stream.of("book", "pen", "ruler")
        .map(s -> s.length()) // String::length
        .forEach(System.out::print);// 435
```

Intermediate Operations - sorted

SYNERGETICS
GET IT RIGHT

- sorted() returns a stream with the elements sorted.
- Just like sorting arrays, Java uses natural ordering unless we provide a comparator
- sorted() is a stateful intermediate operation; it needs to see all of the data before it can sort it.

Primitive Stream

- As opposed to Stream<T> e.g., Stream<Integer>, Stream<Double> and Stream<Long>; Java actually provides other stream classes that you can use to work with primitives:
  - IntStream for primitive types int, short, byte and char
  - DoubleStream for primitive types double and float
  - LongStream for primitive type long

Creating Primitive Stream

```
int[] ia     = {1,2,3};
double[] da = {1.1, 2.2, 3.3};
long[] la    = {1L, 2L, 3L};

IntStream iStream1       = Arrays.stream(ia);
DoubleStream dStream1    = Arrays.stream(da);
LongStream lStream1      = Arrays.stream(la);
System.out.println(iStream1.count() + ", " +
        dStream1.count() + ", " + lStream1.count());// 3, 3, 3

IntStream iStream2       = IntStream.of(1, 2, 3);
DoubleStream dStream2    = DoubleStream.of(1.1, 2.2, 3.3);
LongStream lStream2      = LongStream.of(1L, 2L, 3L);
System.out.println(iStream2.count() + ", " +
        dStream2.count() + ", " + lStream2.count());// 3, 3, 3
```

Creating Primitive Stream

- The primitive streams, in addition to containing many of the Stream methods, also contain specialized methods for working with numeric data.
- The primitive streams know how to perform certain common operations automatically e.g., min(), max(), sum() and average().

Optional class

- Think of an Optional as a box that may or may not be empty.
- Before Java 8, programmers would return null instead of Optional . Returning an Optional is now a clear way for the API to state that there might not be a value in there (and that the programmer must deal with that).
- In addition to Optional<T>, there are Optional ’s for the primitive types:
  - OptionalDouble , OptionalInt and OptionalLong

Optional Example

```
// a long way to calculate average (just for showing Optional)
public static Optional<Double> calcAverage(int... scores){
    if(scores.length == 0) return Optional.empty();
    int sum=0;
    for(int score:scores) sum += score;
    return Optional.of((double)sum / scores.length);
}
```

Optional Example

SYNERGETICS
GET IT RIGHT

```
Optional<Double> optAvg = calcAverage(50, 60, 70);
 // if you do a get() and the Optional is empty you get:
 //    NoSuchElementException: No value present
 // boolean isPresent() protects us from that.
 if(optAvg.isPresent()){
     System.out.println(optAvg.get()); // 60.0
 }
 // void ifPresent(Consumer c)
 optAvg.ifPresent(System.out::println);// 60.0
 // T orElse(T t)
 System.out.println(optAvg.orElse(Double.NaN)); // 60.0

 Optional<Double> optAvg2 = calcAverage();// will return an empty Optional
 System.out.println(optAvg2.orElse(Double.NaN)); // NaN
 // T orElseGet(Supplier<T> s)
 System.out.println(optAvg2.orElseGet(() -> Math.random())); // 0.8524556508038182
```

Parallel Stream

- All of the streams thus far have been sequential streams i.e., the streams have processed the data one element at a time.
- Parallel streams can process elements in a stream concurrently i.e., at the same time.
- Java achieves this by splitting the stream up into sub streams and then the pipeline operations are performed on the sub streams concurrently (each sub stream has its own thread).

Parallel Stream

- To make a stream parallel, we can use the parallel() or parallelStream methods.
- parallel() is available in Stream<T>
- parallelStream () is defined in the Collection<E> interface

Parallel Stream

- Be careful if order is important, as the order of thread completion will determine the final result (not the order in the original collection).

```
public static void sequentialStream() {
    Arrays.asList("a", "b", "c") // create List
            .stream()    // stream the List
            .forEach(System.out::print);// abc
}
public static void parallelStream() {
    Arrays.asList("a", "b", "c") // create List
            .stream()    // stream the List
            .parallel()
            .forEach(System.out::print);// bca
}
```

Java8-DATE-TIME-API

SYNERGETICS
GET IT RIGHT

- New date-time API is introduced in Java 8 to overcome the following drawbacks of old date-time API :
- **Not thread safe :** Unlike old java.util.Date which is not thread safe the new date-time API is *immutable* and doesn't have setter methods.
- **Less operations :** In old API there are only few date operations but the new API provides us with many date operations.
- Java 8 under the package java.time introduced a new date-time API, most important classes among them are :
- **Local :** Simplified date-time API with no complexity of timezone handling.
- **Zoned :** Specialized date-time API to deal with various timezones.

Java8-DATE-TIME-API

SYNERGETICS
GET IT RIGHT

- **LocalDate/LocatTime** and **LocalDateTime API :** Use it when time zones are NOT required.
- **Period** and **Duration** classes :

  Period : It deals with *date* based amount of time.

  Duration : It deals with *time* based amount of time.

SYNERGETICS

GET IT RIGHT

Thank You

Educate | Advise | Implement | Manage